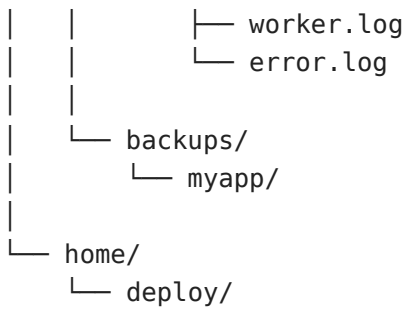


Yes — you mean **production deployment layout and separation of concerns on the VPS**, not programming style.

For a serious production system, I would strongly recommend separating **application code, configuration, database storage, user-generated data, logs, secrets, and deployment releases** rather than putting everything under something like `/home/user/public_html`.

A clean Linux VPS layout would look roughly like this:

```
/
├── etc/
│   └── myapp/
│       ├── backend.env
│       ├── frontend.env
│       └── app.conf
├── srv/
│   └── myapp/
│       ├── frontend/
│       │   ├── current -> releases/20260827_134500
│       │   ├── releases/
│       │   │   ├── 20260820_101500/
│       │   │   ├── 20260825_163000/
│       │   │   └── 20260827_134500/
│       │   └── shared/
│       ├── backend/
│       │   ├── current -> releases/20260827_134500
│       │   ├── releases/
│       │   │   ├── 20260820_101500/
│       │   │   └── 20260827_134500/
│       │   └── shared/
│       └── scripts/
│           ├── deploy.sh
│           ├── rollback.sh
│           └── backup.sh
├── var/
│   ├── lib/
│   │   └── myapp/
│   │       ├── uploads/
│   │       ├── documents/
│   │       ├── exports/
│   │       └── cache/
│   └── log/
│       └── myapp/
│           └── backend.log
```



The conceptual separation is more important than the exact directory names.

Application code

Put your deployed application code somewhere like:

```
/srv/myapp/
```

or:

```
/var/www/myapp/
```

For a React + Node/Laravel/etc. application, I prefer `/srv` because it makes the distinction clearer:

```
/srv/myapp/frontend
/srv/myapp/backend
```

The source code should be treated as **replaceable**.

You should be able to delete a deployed release and redeploy it without losing customer data.

That means things such as these should **never live permanently inside the application release directory**:

```
uploads/  
profile-images/  
invoices/  
documents/  
database/  
backups/  
.env
```

1. Use release directories for deployment

For the seamless updates you were asking about earlier, use:

```
/srv/myapp/backend/  
└─ current -> releases/20260827_134500  
└─ releases/
```

```
|   └─ 20260826_190000/
|   └─ 20260827_090000/
|   └─ 20260827_134500/
└─ shared/
```

A deployment creates:

```
releases/20260827_150000/
```

You completely prepare that release:

```
npm install
npm run build
database migrations
configuration validation
health check
```

Then switch:

```
ln -sf /srv/myapp/backend/releases/20260827_150000 \
    /srv/myapp/backend/current
```

Your Nginx/Apache/PM2/systemd configuration always references:

```
/srv/myapp/backend/current
```

This gives you **atomic deployment**.

If the deployment fails:

```
current
```

continues pointing to the old version.

Rollback is simply changing the symlink back.

2. Configuration belongs outside the code

Do not keep production configuration mixed with releases.

Instead:

```
/etc/myapp/
└─ backend.env
└─ frontend.env
└─ database.env
└─ redis.env
```

For example:

```
/etc/myapp/backend.env
```

could contain:

```
NODE_ENV=production
PORT=3001
```

```
DB_HOST=127.0.0.1
DB_PORT=5432
DB_NAME=myapp
DB_USER=myapp_app
DB_PASSWORD=*****
```

```
REDIS_HOST=127.0.0.1
```

Your repository can contain:

```
.env.example
```

but **never the real production .env**.

An even stronger production setup uses a secret manager, but `/etc/<application>/` with strict Linux permissions is perfectly reasonable for a VPS.

For example:

```
chown root:myapp /etc/myapp/backend.env
chmod 640 /etc/myapp/backend.env
```

3. Database files should never be inside the application

This is particularly important.

Do **not** do this:

```
/srv/myapp/backend/database/postgres/
```

The database server should own its own persistent storage.

For PostgreSQL, commonly:

```
/var/lib/pgsql/
```

or containerized:

```
/var/lib/myapp/postgres/
```

Architecture:

React

↓

Backend API

↓

PostgreSQL

Not:

React → directly to PostgreSQL

The browser should never know:

DB password

DB port

DB credentials

SQL connection string

Only your backend communicates with the DB.

4. User data must be a separate persistent layer

This is one of the most frequently violated rules.

Suppose users upload:

- profile pictures
- PDFs
- invoices
- assignments
- attachments
- videos
- reports

Don't store them under:

`/srv/myapp/backend/current/uploads/`

because the next deployment may replace the directory.

Instead:

`/var/lib/myapp/uploads/`

For example:

```
/var/lib/myapp/  
├─ uploads/  
│   ├─ avatars/  
│   ├─ documents/  
│   ├─ images/  
│   └─ videos/  
└─ generated/
```

```
|   ├── invoices/
|   ├── reports/
|   └── exports/
```

Then your application references that location.

Alternatively:

```
/srv/myapp/shared/uploads
```

and each release gets a symlink:

```
release/uploads -> /srv/myapp/shared/uploads
```

This gives you:

```
CODE      → disposable
DATABASE  → persistent
UPLOADS   → persistent
CONFIG    → persistent
```

That distinction is fundamental.

5. Logs must also live outside releases

Don't allow PM2 applications to produce multi-gigabyte logs inside arbitrary application directories.

Use:

```
/var/log/myapp/
```

For example:

```
/var/log/myapp/
├── api.log
├── api-error.log
├── worker.log
├── payments.log
└── audit.log
```

Then configure **logrotate**.

For example:

```
/etc/logrotate.d/myapp
```

Typical policy:

```
daily
rotate 14
```

compress
missingok
notifempty

So you don't eventually get:

backend-error.log 12 GB
worker.log 8 GB

and exhaust disk space.

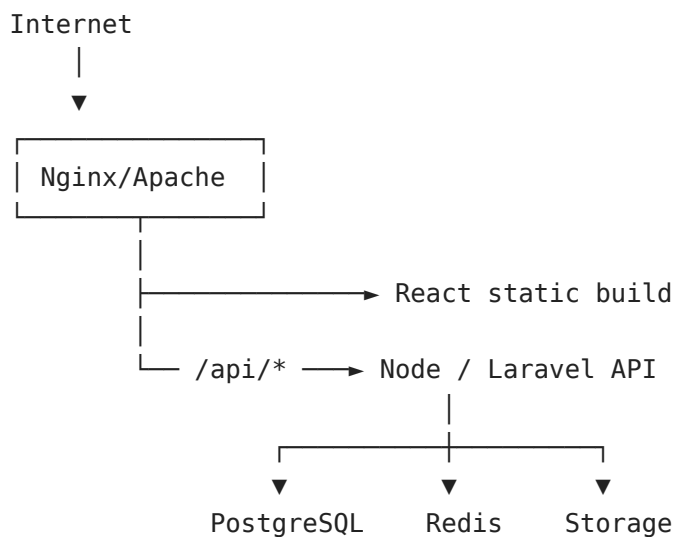
6. Separate runtime processes

For a moderately large project:

Frontend
Backend API
Background workers
Scheduler
Database
Redis
Reverse proxy

should conceptually be independent.

For example:



If using Node, I would normally use **systemd** or a carefully configured PM2 service.

For important infrastructure, systemd gives you cleaner OS-level supervision:

/etc/systemd/system/myapp-api.service
/etc/systemd/system/myapp-worker.service

7. Separate permissions and users

Another important practice is: **the web application should not run as root.**

Create a dedicated account:

```
myapp
```

Potentially also:

```
deploy
```

Then:

```
root
├── controls OS/config
├──
├── deploy
│   ├── deploys releases
│   └──
└── myapp
    └── runs application
```

Your Node process shouldn't have permission to modify:

```
/etc/
/usr/
/root/
```

And ideally it shouldn't even have permission to modify its own application source.

It only needs write access to locations such as:

```
/var/lib/myapp/uploads
/var/log/myapp
```

This substantially limits damage if the application is compromised.

8. Frontend structure

For React/Vite:

```
/srv/myapp/frontend/
├── current -> releases/20260827_134500
└── releases/
    ├── 20260826_100000/
    ├── 20260827_134500/
    └── dist/
```

Nginx might serve:


```
/srv/myapp/frontend/current/dist
```

The frontend server does **not need**:

```
src/  
node_modules/  
.git/  
tests/
```

in the publicly served directory.

Ideally production serves only the compiled artifact:

```
dist/
```

9. Backend structure

For example:

```
/srv/myapp/backend/current/  
├─ dist/  
├─ node_modules/  
├─ package.json  
└─ package-lock.json
```

The backend source may look like:

```
src/  
├─ controllers/  
├─ services/  
├─ repositories/  
├─ models/  
├─ middleware/  
├─ validators/  
├─ routes/  
└─ config/
```

But that is application architecture.

The VPS architecture remains:

code	/srv/myapp
config	/etc/myapp
data	/var/lib/myapp
logs	/var/log/myapp
backups	/var/backups/myapp

That convention is an excellent rule to remember.

10. Database user separation

Don't connect your application using:

```
postgres
root
```

Create an application-specific DB user:

```
myapp_app
```

For example:

```
PostgreSQL
|
|— postgres          ← administration only
|
|— myapp_app         ← normal application
|
|— myapp_migrations  ← optional deployment account
|
|— myapp_readonly    ← reporting/analytics
```

For higher-security systems, separating migration privileges from runtime privileges is particularly useful.

The API account might have:

```
SELECT
INSERT
UPDATE
DELETE
```

but **not**:

```
DROP DATABASE
CREATE ROLE
SUPERUSER
```

11. Database migrations belong with releases

Suppose:

```
Release A
DB schema 15
```

```
Release B
DB schema 16
```

Deployment should execute controlled migration scripts:

```
migrations/  
├─ 001_create_users.sql  
├─ 002_add_roles.sql  
└─ 003_add_profile_image.sql
```

Never manually modify production DB tables whenever a developer needs a field.

All schema changes should be version-controlled.

This becomes especially important when multiple developers are working on the same product.

12. Backups must be outside the application

Never:

```
public_html/backup.sql
```

or:

```
/srv/myapp/current/database.sql
```

Instead:

```
/var/backups/myapp/
```

For example:

```
/var/backups/myapp/  
├─ database/  
├─ uploads/  
└─ configs/
```

And ideally the backups are then copied **off the VPS**.

Otherwise:

| VPS failure = production + backup failure simultaneously.

My preferred structure for your type of VPS

For the kind of React + backend projects you have been discussing, I would standardize every project like this:

```
/srv/apps/  
├─  
├─ project-a/  
│   └─ frontend/
```

```

|   |   |─ current
|   |   └─ releases/
|   |
|   └─ backend/
|       └─ current
|       └─ releases/
|
└─ project-b/
    └─ frontend/
    └─ backend/
|
└─ project-c/
    └─ frontend/
    └─ backend/

```

```

/etc/apps/
└─ project-a/
└─ project-b/
└─ project-c/

```

```

/var/lib/apps/
└─ project-a/
    └─ uploads/
    └─ generated/
└─ project-b/
└─ project-c/

```

```

/var/log/apps/
└─ project-a/
└─ project-b/
└─ project-c/

```

```

/var/backups/apps/
└─ project-a/
└─ project-b/
└─ project-c/

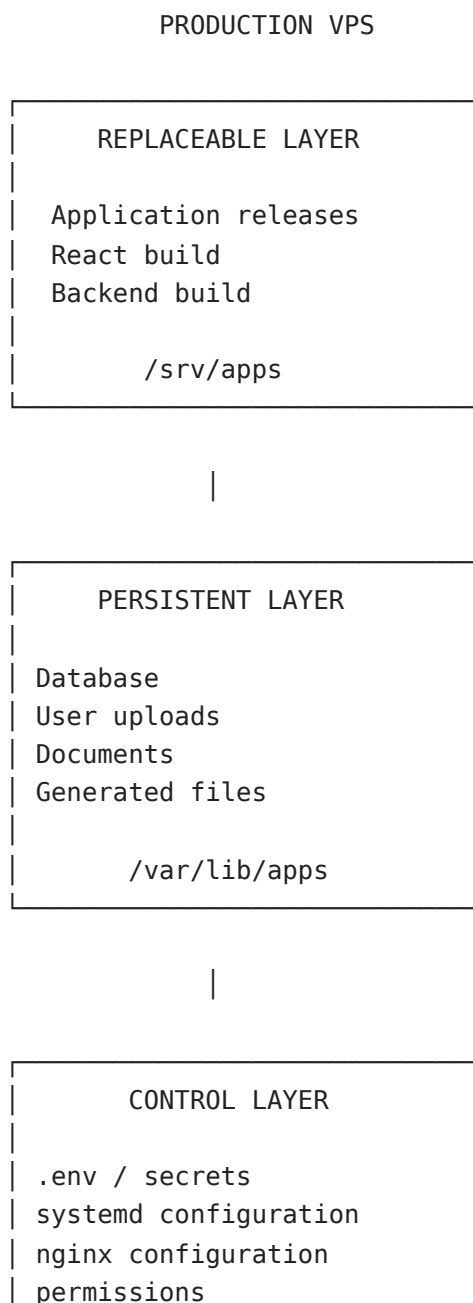
```

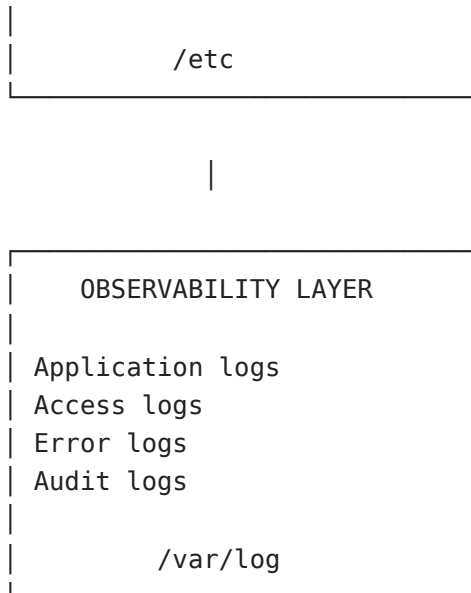
Then the rule for your development team becomes extremely simple:

Component	Location	Persistent?
Application releases	/srv/apps/<app>/	No
Production configuration	/etc/apps/<app>/	Yes
Secrets	/etc/apps/<app>/	Yes

Component	Location	Persistent?
User uploads	/var/lib/apps/<app>/	Yes
Database	DB-managed /var/lib/...	Yes
Logs	/var/log/apps/<app>/	Yes
Backups	/var/backups/apps/<app>/	Yes
Deployment scripts	/srv/apps/<app>/scripts/	Yes
Public React build	frontend/current/dist/	No

The most important architectural principle is:





This architecture also solves the **zero/minimal-downtime deployment** question from earlier: new code is built into a completely separate release directory, validated, and only then does current switch atomically to the new release. The database, uploads, configuration and logs are untouched by deployments.